
Supervised Learning: Classification and Regression

Group 72:

Student Name	Student Number	Section 2(%)	Section 3(%)	Section 4(%)
Ecenaz Elverdi	252699	80	10	10
Burak Yenidzhe	253733	10	80	10
Yusuf Eren Kılıç	252653	10	10	80

Contents

1	Introduction	1
1.1	Feature Set and Preprocessing	1
2	Regression Setup and Evaluation	1
2.1	The Feature Transformation	1
2.2	Estimating Regularization: λ Selection and Generalization Error	1
2.2.1	Choosing Lambda	2
2.2.2	Model Interpretation	3
2.2.3	Interpretation of Model Weights	3
3	Regression Analysis	4
3.1	Experimental Setup	4
3.2	Hyperparameter Search	4
3.3	Statistical Validation (Setup II)	5
4	Model Comparison for Multi-Class Classification	6
4.1	Hyperparameter Ranges	6
4.2	Two-Level Cross-Validation Performance	7
4.3	Statistical Model Comparison	7
4.4	Final Ranking	8
5	Logistic Regression Coefficients and Feature Importance	8
6	Conclusion	10
7	AI Usage	10
	References	10

1 Introduction

The primary objective of this regression and classification analysis is to predict concrete compressive strength (measured in MPa) based on eight input features: Cement, Blast Furnace Slag, Fly Ash, Water, Superplasticizer, Coarse Aggregate, Fine Aggregate, and Age [1].

1.1 Feature Set and Preprocessing

Prior to modeling, we applied Standardization. For this regression analysis, we proceed with all 8 original features to maintain interpretability of individual feature effects.

2 Regression Setup and Evaluation

2.1 The Feature Transformation

As we did in the first project, the feature transformation method we choose was Standardization. The reason we choose Standardization is, because there are big gaps between features, like Cement and Superplasticizer. According to the rules of the Standardization, the mean should be 0 and the standard deviation should be 1.

We won't use One-out-of-K encoding and Binarization. One-out-of-K encoding is used when we don't want nominal data to be ordinal and in a situation where data has no relation to each other. Binarization, on the other hand, converts continuous variables into binary values based on a threshold. It serves as a preprocessing step for machine learning techniques used in image classification and detection tasks, which often require binary images as input.

Since our dataset contains continuous numeric features with meaningful magnitudes, standardization on its own is the appropriate preprocessing step for fair regularization and distance-based calculations.

The target variable (Compressive Strength) was kept in its original scale (MPa) without standardization, as standardizing the output would require inverse transformation of predictions and is unnecessary for regression modeling.

2.2 Estimating Regularization: λ Selection and Generalization Error

Regularization is a technique that help preventing overfitting. It adds a penalty term to the loss function that discourages complex models.

$$\text{Loss} = \text{Data Fit Error} + \lambda \times \text{Complexity Penalty} \quad (1)$$

Where Data Fit Error shows how well model fits training data is (e.g., squared error), Complexity Penalty measure of model complexity (e.g., size of weights), and λ (lambda) controls trade-off between fit and complexity.

If we don't regularize the data ($\lambda = 0$), the model will be very flexible, giving low bias and it will be sensitive to training data, causing high variance. This might cause overfitting: the test error becomes high while the training error stays low.

On the other hand, we apply regularization ($\lambda > 0$), then the model will be constrained, making bias slightly higher. It will be less sensitive to training data, causing variance to be lower. With this, we get better generalization, lowering the test error.

2.2.1 Choosing Lambda

Before fitting the regularized model, we must select an appropriate value for the regularization parameter λ . To do this, we evaluate a range of candidate values and estimate their generalization performance using K=10-fold cross-validation. We go with cross validation instead of training error because if we choose λ based on training error, it will always decrease with smaller λ . Change of λ and mean cross validation error can be observed on Table 1.

λ (Lambda)	Mean CV Error (MSE)
0.0010	109.77
0.0021	109.77
0.0043	109.77
0.0089	109.77
0.0183	109.77
0.0379	109.77
0.0785	109.77
0.1624	109.77
0.3360	109.77
0.6952	109.76
1.4384	109.76
2.9764	109.77
6.1585	109.84
12.7427	110.11
26.3665	110.80
54.5559	112.24
112.8838	115.11
233.5721	121.25
483.2930	134.31
1000.0000	158.04

Table 1: Cross-validation error for different regularization strengths λ .

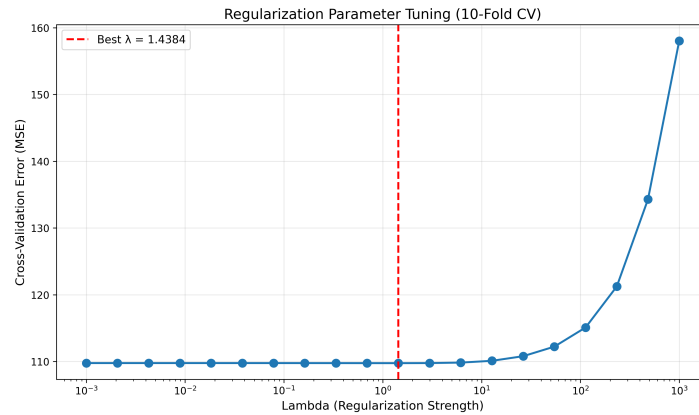


Figure 1: Generalization Error Estimation

As we saw at the Table 1 and Figure 1, $\lambda = 1.4384$ gives the best value with 109.76 cross validation error. With this lambda value, we show the training and test errors for each of the 10 folds. The small gap between train and test error indicates good generalization with minimal overfitting. The consistency across folds validates the robustness of our model selection.

2.2.2 Model Interpretation

The Ridge regression model with optimal $\lambda = 1.4384$ computes the concrete compressive strength using the linear equation:

$$y = w_0 + w_1 x_{\text{cement}} + w_2 x_{\text{slag}} + w_3 x_{\text{flyash}} + w_4 x_{\text{water}} + w_5 x_{\text{superplasticizer}} + w_6 x_{\text{coarse}} + w_7 x_{\text{fine}} + w_8 x_{\text{age}} \quad (2)$$

Where w_0 is the intercept (bias term), $w_1, w_2, \dots, w_8$ are the learned feature weights from Ridge regression, and $x_1, x_2, \dots, x_8$ are the standardized input features.

In addition to these eight weights, the model also includes an intercept term $w_0 = 35.82$ MPa, which represents the baseline prediction when all features are at their average values. Each feature weight shows how the predicted strength changes when that feature increases by one standard deviation.

2.2.3 Interpretation of Model Weights

Cement ($w = +12.26$): The most influential factor. A one standard deviation increase in cement increases strength by approximately 12.26 MPa. Cement is the primary binding agent in concrete.

Blast Furnace Slag ($w = +8.71$): The second-most influential factor. Slag is a supplementary cementitious material that can replace some cement while maintaining strength.

Age ($w = +7.19$): The third-most influential factor. Concrete gains strength over time through cement hydration. Each standard deviation increase in age increases strength by 7.19 MPa.

Fly Ash ($w = +5.41$): Another supplementary cementitious material with a positive effect on strength, though less than slag.

Water ($w = -3.38$): The only negative weight. Higher water content reduces concrete strength, following the fundamental water–cement ratio principle.

Superplasticizer ($w = +1.74$): A minor positive effect. This admixture improves workability and indirectly increases strength.

Coarse and Fine Aggregates ($w \approx +1.2\text{--}1.4$): The least influential factors. Aggregates are primarily structural fillers and have minimal direct effect on strength.

The learned weight structure aligns well with concrete science: cement is most important, water has a negative relationship, and age captures strength gain. The negative weight for water correctly reflects the water-cement ratio principle. The $10\times$ difference between cement and aggregates reflects their relative importance. This physical interpretability shows the model learned meaningful relationships.

3 Regression Analysis

3.1 Experimental Setup

The objective of this section was to rigorously compare three regression models on the Concrete Compressive Strength dataset: a Baseline model, a Ridge Regression model (LR), and an Artificial Neural Network (ANN). To ensure a fair and unbiased evaluation, we implemented a two-level cross-validation with $K_1 = K_2 = 10$ folds. The inner loop handled hyperparameter optimization, while the outer loop provided an independent estimate of each model’s generalization ability. All models were trained on identical data splits to enable valid statistical testing later.

3.2 Hyperparameter Search

The baseline model predicts the mean target value of the training set for all test samples. Although extremely simple, it serves an important role as a reference point, showing how much improvement can be attributed purely to learning patterns in the data rather than to chance or data scaling. In other words, if a more complex model cannot outperform this baseline, it indicates that the additional complexity does not lead to better generalization.

To control model complexity and mitigate overfitting, we used Ridge Regression. The regularization strength λ was tuned logarithmically across $\lambda \in [10^{-3}, 10^3]$ using 20 evenly spaced values. This approach allowed identifying both the under and over regularization regimes.

The ANN used a single hidden layer with a tunable number of neurons $h \in 20, 50, 80, 110, 140$. The network was trained using the LBFGS optimizer for up to 3000 iterations, with the ReLU activation

function, which allows the model to learn non-linear relationships efficiently while avoiding vanishing gradient problems. This setup provided a stable and computationally applicable way to explore different model complexities.

The results of the two-level cross-validation are summarized in Table 2. Each outer fold was evaluated using the optimal hyperparameters (h_i^*, λ_i^*) determined in the corresponding inner loop. The reported E_i^{test} values represent the Mean Squared Errors on the test partitions of the outer folds.

Table 2: Two-level cross-validation results.

Outer fold i	h_i^*	λ_i^*	ANN E_i^{test}	LR E_i^{test}	Baseline E_i^{test}
1	80	1.434	27.94	101.22	268.48
2	50	1.434	28.29	86.28	247.72
3	80	0.695	20.34	135.57	299.11
4	80	1.434	37.64	133.60	371.86
5	140	1.434	40.60	142.39	263.15
6	110	1.434	11.66	180.84	304.37
7	80	2.977	28.86	90.18	302.58
8	80	0.336	19.51	89.34	261.80
9	50	0.162	29.61	105.18	244.91
10	50	0.001	21.98	105.70	227.35
Mean	–	–	26.64	109.83	279.13

At a glance, the table shows that the error values vary moderately between folds, reflecting natural differences in the train-test splits. The optimal λ values for Ridge Regression are generally around 1.43, which is consistent with the value found in previous part, confirming that a moderate level of regularization works best for this dataset. For the ANN, the optimal hidden-layer size h most frequently occurs around 80, indicating that this configuration provides a good balance between model capacity and generalization. Overall, the table suggests that both models adapt well across folds and that the chosen hyperparameter ranges were appropriate.

3.3 Statistical Validation (Setup II)

Although the Mean Squared Error values provide a general overview of model performance, they do not by themselves establish whether the observed differences are statistically significant. Since all models were evaluated on overlapping training sets across the K -fold cross-validation, the test results are not independent, making a standard paired t-test invalid.

To properly account for this dependency, we applied the correlated t-test (Setup II) as described in

the course notes. This method adjusts the variance estimate to reflect the correlation between folds and therefore provides a more accurate test of significance when models share training data. The test was conducted at a significance level of $\alpha = 0.05$, comparing the models pairwise is shown in Table 3.

Table 3: Pairwise statistical comparison results using the correlated t-test (Setup II) at $\alpha = 0.05$.

Comparison	Mean Difference ($\hat{r}$)	95% CI	p-value	Conclusion
ANN vs. LR	83.19	[63.65, 102.73]	4.89×10^{-6}	ANN significantly better
ANN vs. Baseline	252.49	[209.66, 295.31]	3.1×10^{-7}	ANN significantly better
LR vs. Baseline	169.30	[129.93, 208.67]	4.5×10^{-6}	LR significantly better

The statistical comparison (Setup II) confirms that all observed performance differences are statistically significant ($p < 0.001$). The ANN model outperforms both LR and the Baseline by a large margin, as indicated by strictly positive confidence intervals and very low p-values. The LR model also performs significantly better than the Baseline, suggesting that it captures some meaningful linear trends in the data. However, since the ANN achieves the lowest MSE and a substantial improvement over both alternatives, it can be considered the most suitable model for this regression task.

4 Model Comparison for Multi-Class Classification

In the first part of the project, the concrete compressive strength was divided into three categories of low, medium, and high strength. This formulation leads to a multi-class classification task. In this section, we compare the performance of three models: multinomial logistic regression, K-Nearest Neighbors (KNN), and a baseline classifier. KNN was included due to its simplicity and flexibility.

Hyperparameter tuning and model comparison were performed using a two-level cross-validation procedure. The outer loop consisted of 10 folds for estimating the generalization error, while a 5-fold inner loop was used to select optimal hyperparameters. This ensures that no information from the test partition leaks into the model selection process.

4.1 Hyperparameter Ranges

In logistic regression problem, the regularization parameter λ controls the complexity of the model. A larger value of λ corresponds to stronger regularization. We evaluated the following range of regularization strengths: $\lambda \in \{1000, 100, 10, 1, 0.1, 0.01\}$.

For KNN, the complexity-controlling parameter is the number of neighbors k , with smaller values producing more flexible decision boundaries. We choosed the range $k \in \{1, 3, 5, 7, 9, 11, 15, 21, 31, 51\}$.

The baseline classifier predicts the most frequent class in each outer training fold.

4.2 Two-Level Cross-Validation Performance

Across the 10 outer folds, KNN was the most frequently selected $k = 1$ and achieved the lowest generalization error in 7 out of 10 folds. This indicates that the dataset favors a highly flexible nearest-neighbor decision boundary. Logistic regression yielded the best performance in the remaining 3 folds. The baseline performed consistently worst. Table 4 summarizes errors and accuracies on all folds.

Table 4: Outer-fold results from nested cross-validation for Baseline, KNN, and Logistic Regression.

Fold	x_i			E_{test}			Accuracy		
	Baseline	KNN	LogReg	Baseline	KNN	LogReg	Baseline	KNN	LogReg
0		1	0.01	0.6832	0.2079	0.3267	0.3168	0.7921	0.6733
1		1	0.1	0.6337	0.2673	0.2673	0.3663	0.7327	0.7327
2		1	0.001	0.6634	0.2178	0.2871	0.3366	0.7822	0.7129
3		1	0.001	0.6832	0.2772	0.2970	0.3168	0.7228	0.7030
4		1	0.01	0.6139	0.1980	0.1881	0.3861	0.8020	0.8119
5		1	0.1	0.6500	0.2300	0.3500	0.3500	0.7700	0.6500
6		1	0.001	0.5800	0.2800	0.2600	0.4200	0.7200	0.7400
7		1	0.1	0.6400	0.1900	0.2700	0.3600	0.8100	0.7300
8		1	0.01	0.7600	0.2600	0.2800	0.2400	0.7400	0.7200
9		1	0.1	0.6400	0.2300	0.3200	0.3600	0.7700	0.6800

4.3 Statistical Model Comparison

To evaluate whether the observed differences in accuracy were statistically significant, we conducted pairwise McNemar tests with $\alpha = 0.05$. The following summarizes the contingency tables, effect estimates, confidence intervals, and p-values. Here, confidence interval is the estimated difference in accuracy.

KNN vs Logistic Regression

$$\theta = -0.0488, \quad 95\% \text{ CI} = [-0.0813, -0.0162], \quad p = 0.0041.$$

The negative theta indicates KNN performs approximately 4.9% better than Logistic Regression. The entirely negative confidence interval confirms KNN's consistent superiority. The statistically significant p-value ($p < 0.01$) strongly supports this conclusion.

KNN vs Baseline

$$\theta = 0.4189, \quad 95\% \text{ CI} = [0.3806, 0.4565], \quad p = 6.24 \times 10^{-80}.$$

KNN substantially outperforms the Baseline model by approximately 41.9%. The confidence interval is positive and far from zero. This indicates high certainty. The extremely small p-value demonstrates overwhelming statistical significance.

Logistic Regression vs Baseline

$$\theta = 0.3701, \quad 95\% \text{ CI} = [0.3290, 0.4106], \quad p = 2.87 \times 10^{-58}.$$

Logistic Regression significantly outperforms the Baseline by approximately 37%. This is supported by positive confidence interval and approximately zero p-value.

4.4 Final Ranking

As a conclusion, all pairwise tests reject the null hypotheses. KNN is statistically significantly better than Logistic Regression, which is in turn significantly better than the Baseline. Therefore, KNN is the recommended model for this problem.

5 Logistic Regression Coefficients and Feature Importance

Multinomial logistic regression predicts class probabilities using the softmax function:

$$P(y = c \mid x) = \frac{\exp(w_c^\top x + b_c)}{\sum_{k=1}^3 \exp(w_k^\top x + b_k)}.$$

The predicted class is the one with the highest probability,

$$\hat{y} = \arg \max_c P(y = c \mid x).$$

This creates linear decision boundaries between the three strength categories. To select the optimal regularization strength for logistic regression, we performed a nested cross-validation procedure. The inner loop was used for hyperparameter tuning, and the outer loop for estimating the generalization error. As a result $\lambda = 0.1$ is found as the best value (Table 5). Using this hyperparameter, we trained the final multinomial logistic regression model on the full training dataset. The estimated coefficient matrix can be seen on Figure 2.

To quantify feature importance, we computed the mean absolute coefficient across classes. Table 6 reports the results.

Age is by far the most influential feature in separating the strength classes. Cement and slag also play an important role. Aggregate content and water contribute comparatively little. These findings align

Table 5: Mean 10-fold cross-validation error for different values of λ .

λ	Mean CV Error
1000	0.421851
100	0.346198
10	0.323416
1	0.286594
0.1	0.279614
0.01	0.282594

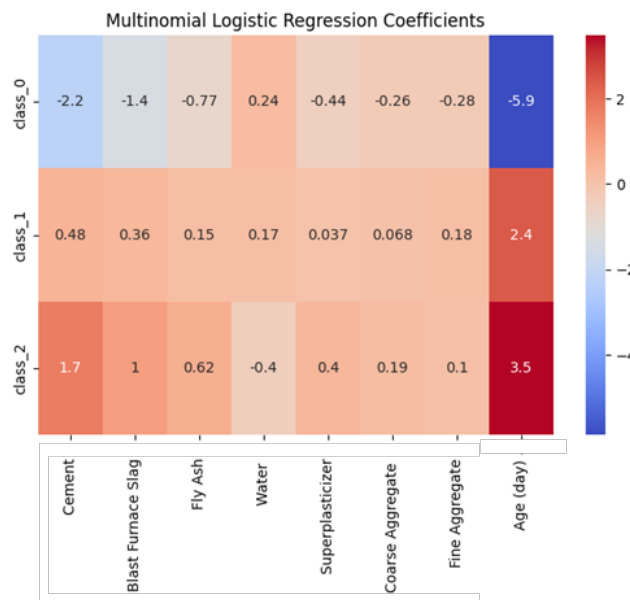


Figure 2: Multinomial logistic regression coefficients

Table 6: Feature importance based on mean absolute logistic regression coefficients.

Feature	Importance
Age	3.91
Cement	1.48
Slag	0.93
Fly Ash	0.51
Superplasticizer	0.29
Water	0.27
Fine Aggregate	0.19
Coarse Aggregate	0.17

well with the results from the regression section, indicating that both the continuous and categorical models depend on similar underlying physical factors.

6 Conclusion

In this report, we evaluated models for both regression and classification of concrete compressive strength. For the regression task, the ANN outperformed both Ridge Regression and the baseline, achieving a statistically significant reduction in mean squared error. This strongly suggests that the relationships between the concrete's components and its final strength are highly non-linear. In the classification task, the KNN model was found to be statistically superior to multinomial logistic regression. Moreover, during the analyzes of logistic regression's coefficients, we observed that the same features were favored with the regression task. We identified Age and Cement as the most dominant predictors of strength. At the previous research introduced by Yeh (1998) [2] that used the same dataset, author compared classical regression models with artificial neural networks. At the end, ANN results outperformed the traditional regression models. Our findings align well with the results of that study since we also obtained the best regression results by using an ANN.

7 AI Usage

In this project, we made limited use of AI tools primarily for language editing and sentence refinement.

References

- [1] I.-C. Yeh. *Concrete Compressive Strength [Dataset]*. Accessed: 2025-11-11. 1998. URL: <https://archive.ics.uci.edu/dataset/165/concrete-compressive-strength>.
- [2] I.-C. Yeh. "Modeling of strength of high-performance concrete using artificial neural networks". In: *Cement Concrete Research* 28.12 (1998), pp. 1797–1808. DOI: 10.1016/S0008-8846(98)00165-3.